Logical pointers realize the rule that pointers derived from one object can never be used to modify other objects. This is achieved by making it impossible to change the value of l: Pointer arithmetic only affects the offset o.

Physical Pointers. As we have seen in Section 2.4, tracking objects in a pointer obtained by an integer-to-pointer cast is not viable because (1) some addresses may be within bounds of multiple objects, and (2) it prevents reordering of instructions. Hence we introduce physical pointers, roughly corresponding to pointers in a flat memory model.

A physical pointer $\mathsf{Phy}(o, s, I, cid)$ consists of an offset o within the address space s (i.e., the physical address), as well as two additional fields I and cid to restrict the set of objects the pointer can access. This allows us to recover alias analysis precision, and it enables several of the optimizations allowed by the C and C++ standards. Field I is a set of physical addresses that corresponds to the `inb` field we used to specify deferred bounds checking in Section 3.1. When the pointer is dereferenced, each address in I must be inbounds of the same object as o. Field cid is a call id, which corresponds to the time stamp when the pointer was passed as argument to a function, or **None** if the pointer did not originate from an argument. The intent is to show that pointers received as arguments do not alias locally allocated objects, e.g.:

```
int f(int *p) {
    int a = 0;
    if (&a == p)
        *p = 1;
    return a;  // returns 0 or 1?
}
```

Since a physical pointer can access any object, if there was no call id restriction, this function could return either `0` or `1`. However, since `p` has a call id of a function call still on the call stack, it cannot access any object created after the call.

The motivation to have an indirection in the time stamp of the call time in the pointer (cid indexes in memory M to retrieve the time stamp) is to support escaping pointers. Escaped physical pointers should behave as their cids being **None** after termination of the function call. This supports moving function calls across other function calls. If a function stores a pointer received by argument in a global variable, we did not want to have to record that fact and change all such pointers when the function returns. This way we only need to change the mapping between cid and time stamp on function return (set it to **None**).

4.2 Address Spaces

LLVM uses address spaces to represent distinct memories, and our memory model also supports this feature. For example, a machine might use one address space for the CPU and another for the GPU, or one address space for code and another for data. Since both memories may have overlapping address ranges (e.g., they may both use addresses in the range $[0, 2^{64})$), the address space field in pointers is used to disambiguate between the two.

The main memory of the CPU is assigned the address space zero. It is possible that a physical memory region is mapped into multiple address spaces. In this case the application can use an address cast instruction to convert pointers between address spaces.

A consequence of possible overlapping of address spaces is that pointers belonging to different address spaces may alias. To improve precision of alias analysis, we parameterize our model by the overlap.